

# Discussion Module:

DSE Search, Scalars

Scalars: Non-tokenized column values,  
letters, numbers

“How are you?”  
Broken. Useless. Alone. Clueless.  
fused. Betrayed. Fragile. On the ve  
tears. Depressed. Anxious. About  
ak down. Really give up. Pathet  
nnoying. I’m just a burden. Distan  
onely. Bitter. Heartbroken. Lonely  
cted. Crushed. I feel like I’m goin  
st fall apart at any moment. Empty  
Defeated. Never good enough.

# Discussion Lab:

Matching pairs – Match the query on the right with the scalar, non-scalar label on the left

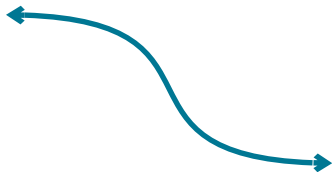


scalars

spatial  
GeoSpatial

text

# Discussion Lab: Queries- Scalar, Non-scalar



scalars



- ☐ `col1 = x AND col2 = y`
- ☐ `col1 = x OR col2 = y`
- ☐ `col1 = x ORDER BY col2`
- ☐ `col1 > x AND col2 < y`
- ☐ `col1 LIKE 'Dav*'`
- ☐ `solr_query = '{"q":"col4:stARrbUx"}'`
- ☐ `solr_query = '{"q":"col4:stARrbUcks"}'`

# End of Discussion Lab:

# docValues: field type property, and/or field property

- [https://lucene.apache.org/solr/guide/6\\_6/docvalues.html#DocValues-EnablingDocValues](https://lucene.apache.org/solr/guide/6_6/docvalues.html#DocValues-EnablingDocValues)
- Supported column types
  - StrField, UUIDField
  - Any Trie\*; numeric fields, date fields, EnumField
  - Boolean
  - Int, Long, Float, Double, DatePointField
- *TextField with KeywordTokenizer*

## why

/ˈ(h)wɪ/ ⓘ

*adverb*

1. for what reason or purpose.

"why did he do it?"

*synonyms:* how come, for what reason, for what purpose, what for  
*archaic* **wherefore**

"I know you're quitting school, but why?"

*adverb*

1. (with reference to a reason) on account of which; for which.

"the reason why flu shots need repeating every year is that the v

*exclamation*

1. expressing surprise or indignation.

"Why, that's absurd!"

2. used to add emphasis to a response.

""You think so?" "Why, yes.""

*noun*

1. a reason or explanation.

"the whys and wherefores of these procedures need to be explai



# docValues: Why-

The **standard** way that **Solr** builds the **index** is with an **inverted index**. This style builds a **list of terms found in all the documents** in the index and **next to each term is a list of documents** that the term appears in (**as well as how many times the term appears in that document**). This makes search very fast - since users search by terms, having a ready list of term-to-document values makes the query process faster.

For **other features** that we now commonly associate with search, **such as sorting, faceting, and highlighting**, this approach is **not very efficient**. The **faceting engine**, for example, must **look up each term** that appears in **each document** that will **make up the result set** and **pull the document IDs** in order to **build the facet list**. In Solr, this is maintained **in memory**, and can be **slow to load** (depending on the number of documents, terms, etc.).

In Lucene 4.0, a new approach was introduced. **DocValue fields** are now **column-oriented** fields with a **document-to-value mapping** built at index time. This approach promises to **relieve** some of the **memory requirements** of the fieldCache and **make** lookups for faceting, **sorting**, and grouping much **faster**.

Source: [https://lucene.apache.org/solr/guide/6\\_6/docvalues.html#DocValues-EnablingDocValues](https://lucene.apache.org/solr/guide/6_6/docvalues.html#DocValues-EnablingDocValues)

# docValues: Why, short version-



## A shorter version ?

- The column value is not stored in a Tf-Idf index.
- Tokens are stored in the Tf-Idf index.
- You can reassemble the column value from tokens, but ..



# StrField versus TextField: (2012 article)

“A text field is a sequence of terms that has been tokenized while a string field is a single term (although it can also be multivalued).

“**Punctuation and spacing is ignored for text fields. Text tends to be lowercased, stemmed, and even stop words removed.** You tend to **search text using a handful of keywords whose exact order is not required**, although quoted phrases can be used as well. Fuzzy queries can be done on individual terms (words). Wild cards as well.

“**String fields** are literal character strings with all punctuation, spacing, and case preserved. **Anything other than exact match is done using wild cards**, although (a fuzzy query is also supported).

“String fields are useful for facets and filter queries or display. Text fields are useful for keyword search.

“Synonyms are a token filtering, which applies to text fields, not string fields.

“A fuzzy query would not work properly for a synonym expansion in which some of the terms are phrases, but should otherwise work for a text field term.

Source: <http://lucene.472066.n3.nabble.com/Difference-between-textfield-and-strfield-td3986916.html>



# KeywordTokenizer: the default for TextField

- The non-tokenizer tokenizer-

- *Officially grey per,*

[https://lucene.apache.org/solr/guide/6\\_6/docvalues.html#DocValues-EnablingDocValues](https://lucene.apache.org/solr/guide/6_6/docvalues.html#DocValues-EnablingDocValues)

- docValues this
- Can chain; lowercase filter ??
- Was default < 6.x

## KeywordTokenizer

This tokenizer treats the entire text field as a single token.

**Factory class:** `solr.KeywordTokenizerFactory`

**Arguments:** None

**Example:**

```
<analyzer>
  <tokenizer class="solr.KeywordTokenizerFactory"/>
</analyzer>
```

**In:** "Please, email [john.doe@foo.com](mailto:john.doe@foo.com) by 03-09, re: m37-xq."

**Out:** "Please, email [john.doe@foo.com](mailto:john.doe@foo.com) by 03-09, re: m37-xq."

# End to End Example:

Scalars; StrField, (numerics),  
*TextField with  
KeywordTokenizer*

- DSE cluster, with Search enabled
- Authority to make tables, other
- CQLSH/Studio access-
- All new example; scalars



# Example Scalars .. page 1 of 6

OLD  
OLD  
OLD

```
DROP KEYSPACE ks_7447;  
CREATE KEYSPACE ks_7447 WITH  
REPLICATION =  
    {'class': 'SimpleStrategy', 'replication_factor': 1};  
USE ks_7447;  
CREATE TABLE t3  
(  
    col1        TEXT,  
    col2        TEXT,  
    col3        TEXT,  
    col4        TEXT,  
    col5        TEXT,  
    col6        DOUBLE,  
    col7        INT,  
    col8        DOUBLE,  
    col9        INT,  
    col0        TEXT,  
    PRIMARY KEY ((col1, col2)) );
```

```
CREATE SEARCH INDEX ON t3 WITH  
COLUMNS col1,  
    * { excluded : true };
```



# Example Scalars .. page 2 of 6



```
INSERT INTO t3
(col1, col2, col3, col4, col5,
 col6, col7, col8, col9, col0)
VALUES ('aaa', 'aaa', 'Dave', 'Dave', 'aaa',
 17.76, 17, 17.76, 17, 'aaa');

VALUES ('bbb', 'bbb', 'David', 'David', 'bbb',
 17.092, 17, 17.092, 17, 'bbb');

VALUES ('ccc', 'ccc', 'Davidia', 'Davidia', 'ccc',
 16.003, 16, 16.003, 16, 'ccc');

VALUES ('ddd', 'ddd', 'Sophie', 'Stinky', 'ddd',
 16.00002, 16, 16.00002, 16, 'ddd');
```

# Example Scalars .. page 3 of 6



```
ALTER SEARCH INDEX SCHEMA ON t3
ADD types.fieldtype[@class=
'org.apache.solr.schema.StrField',
@name='StrField53'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3
ADD fieldType[@name='TextField54',
@class='solr.TextField'] WITH
{'"analyzer":{ "tokenizer":{ "class":
"solr.KeywordTokenizerFactory"},
"filter":{ "class": "solr.LowerCaseFilterFactory"}}};
```

```
ALTER SEARCH INDEX SCHEMA ON t3
ADD types.fieldtype[@class=
'org.apache.solr.schema.TrieDoubleField',
@name='TrieDoubleField55'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3
ADD types.fieldtype[@class=
'org.apache.solr.schema.TrieIntField',
@name='TrieIntField56'];
```



# Example Scalars .. page 4 of 6



```
ALTER SEARCH INDEX SCHEMA ON t3 ADD  
fields.field[@name='col3',  
  @type='StrField53', @docValues='true'];  
ALTER SEARCH INDEX SCHEMA ON t3 ADD  
fields.field[@name='col4',  
  @type='TextField54', @docValues='true'];  
ALTER SEARCH INDEX SCHEMA ON t3 ADD  
fields.field[@name='col6',  
  @type='TrieDoubleField55', @docValues='true'];  
ALTER SEARCH INDEX SCHEMA ON t3 ADD  
fields.field[@name='col7',  
  @type='TrieIntField56', @docValues='true'];
```

```
RELOAD SEARCH INDEX ON t3;  
REBUILD SEARCH INDEX ON t3;
```

# Example Scalars .. page 5 of 6



```
SELECT * FROM t3 ORDER BY col6 ASC, col7 DESC, col3 ASC;  
SELECT * FROM t3 WHERE col6 > 12.0 AND col7 < 26;
```

// The statement below fails

```
SELECT * FROM t3 WHERE col6 > 12.0 OR col7 < 26;
```

```
SELECT * FROM t3 WHERE solr_query = 'col6:[12.0 TO *] OR col7:[* TO 26]';  
SELECT * FROM t3  
WHERE solr_query = 'col6:[12.0 TO *] AND col7:[* TO -10]';
```

// The statement below fails

```
SELECT * FROM t3  
WHERE solr_query = 'col6:[12.0 TO *] AND col7:[* TO 26]' ORDER BY col6 ASC;
```

```
SELECT * FROM t3  
WHERE solr_query = '{"q" : "col6:[12.0 TO *] AND col7:[* TO 26]",  
"sort":"col6 ASC, col7 DESC, col3 ASC" }';
```

# Example Scalars .. page 6 of 6



```
SELECT * FROM t3  
WHERE col3 LIKE 'Dav%';
```

**col3**

```
WHERE col3 LIKE 'dav%';
```

```
SELECT * FROM t3  
WHERE col4 LIKE 'Dav%';
```

**col4**

```
WHERE col4 LIKE 'dav%';
```

```
WHERE col4 LIKE '%av%';
```





# End to End Example:

Scalars; StrField, (numerics),  
*TextField* with  
*KeywordTokenizer* Review



# Search predicates: CQL or JSON

## CQL (search predicates)



## JSON (search predicates)

[https://docs.datastax.com/en/datastax\\_enterprise/4.8/datastax\\_enterprise/srch/srchJSON.html](https://docs.datastax.com/en/datastax_enterprise/4.8/datastax_enterprise/srch/srchJSON.html)

[https://docs.datastax.com/en/datastax\\_enterprise/4.8/datastax\\_enterprise/srch/srchCql.html](https://docs.datastax.com/en/datastax_enterprise/4.8/datastax_enterprise/srch/srchCql.html)

```
WHERE solr_query = 'col6:[12.0 TO *]  
OR col7:[* TO 26]';
```

```
WHERE solr_query = '{"q" : "col6:[12.0 TO *  
AND  
col7:[* TO 26]","  
"sort":"col6 ASC, col7 DESC, col3 ASC" }';
```



# CQL Completeness: DSE 6.x, and pre 6.x

- > < <= => = !=
- IS NOT NULL
- IS NULL (Fails: 6.0 and before)
- IN (col in list of static values)
- CONTAINS (Array/collection types: Set, List Map)
- LIKE (StrField, fails on TextField)



# CONTAINS multiValued : field property

- Collections: SET, LIST, MAP

```
CREATE TABLE my_table  
(  
  col1 TEXT,  
  col2 TEXT,  
  col3 TEXT,  
  col4 TEXT,  
  col8 SET<TEXT>,  
  PRIMARY KEY (col1)  
);
```

- Use cases:
  - Tags
  - 32K column value limit
  - (Other)

text

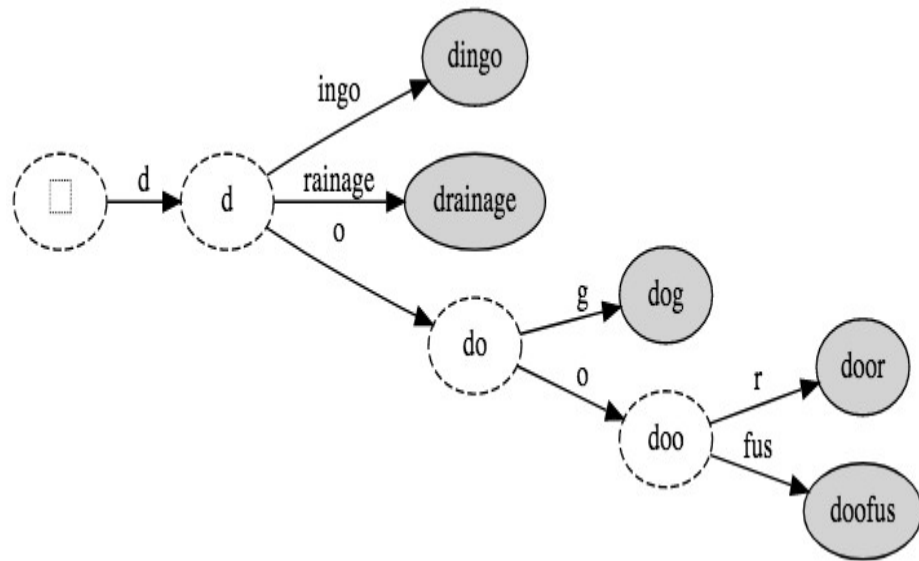
positionIncrementGap

Used for tokenized text only-

Field type property ??

“For multivalued fields, specifies a distance between multiple values, which prevents spurious phrase matches

# Where does the word Trie come from ?



Source: <http://mentaldetritus.blogspot.com/2013/01/grokking-solr-trie-fields.html>  
<http://mentaldetritus.blogspot.com/2013/01/grokking-solr-trie-fields.html>  
<https://en.wikipedia.org/wiki/Trie>

“In computer science, a trie, also called digital tree and sometimes radix tree or prefix tree (as they can be searched by prefixes), is a kind of search tree—an **ordered tree data structure** that is used to store a dynamic set or associative array where the keys are usually strings.

Unlike a binary search tree, no node in the tree stores the key associated with that node; instead, its position in the tree defines the key with which it is associated. **All the descendants of a node have a common prefix** of the string associated with that node, and the root is associated with the empty string.

Values are not necessarily associated with every node. Rather, values tend only to be associated with leaves, and with some inner nodes that correspond to keys of interest~

```
ALTER SEARCH INDEX SCHEMA ON t3
  ADD types.fieldtype[@class=
    'org.apache.solr.schema.TrieDoubleField',
    @name='TrieDoubleField61', @precisionStep='16'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3
  ADD types.fieldtype[@class=
    'org.apache.solr.schema.TrieDoubleField',
    @name='TrieDoubleField62', @precisionStep='4'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3
  DROP fields.field[@name='col6'];
```

```
ALTER SEARCH INDEX SCHEMA ON t3
  ADD fields.field[@name='col6',
    @type='TrieDoubleField61', @docValues='true'];
ALTER SEARCH INDEX SCHEMA ON t3
  ADD fields.field[@name='col8',
    @type='TrieDoubleField62', @docValues='true'];
```

## Trie\* : precisionStep, field type property

- A lower value has more index entries ( > disk size) and faster index processing.
- A higher value has fewer index entries ( < disk size) and slower index processing.
- Default == 8
- 0 (zero) == fastest



# End of Module:





# Additional Content:



# DSE to Solr field type mappings-

## Mapping of Solr types

This table shows the DataStax Enterprise mapping of Solr types to CQL types and Cassandra validators:

| Solr Type         | CQL type      | Cassandra validator | Description                                     |
|-------------------|---------------|---------------------|-------------------------------------------------|
| AsciiStrField     | ascii         | AsciiType           | Indexed as a standard Solr StrField             |
| BCDIntField       | int           | Int32Type           | Binary-coded decimal (BCD) integer              |
| BCDLongField      | bigint        | LongType            | BCD long integer                                |
| BCDStrField       | text, varchar | UTF8Type            | BCD string                                      |
| BinaryField       | blob          | BytesType           | Binary data                                     |
| BoolField         | boolean       | BooleanType         | True (1, t, or T) or False (not 1, t, or T)     |
| ByteField         | int           | Int32Type           | Contains an 8-bit number value                  |
| DateField         | timestamp     | DateType            | Point in time with millisecond precision        |
| DecimalStrField   | decimal       | DecimalType         | Indexed as a standard Solr StrField             |
| DoubleField       | double        | DoubleType          | Double (64-bit IEEE floating point)             |
| EnumField         | text, varchar | UTF8Type            | A closed set having a pre-determined sort order |
| ExternalFileField | text, varchar | UTF8Type            | Values from disk file                           |
| FloatField        | float         | FloatType           | 32-bit IEEE floating point                      |

[https://docs.datastax.com/en/datastax\\_enterprise/5.0/datastax\\_enterprise/srch/solrTypeMapping.html](https://docs.datastax.com/en/datastax_enterprise/5.0/datastax_enterprise/srch/solrTypeMapping.html)

[https://lucene.apache.org/solr/guide/6\\_6/field-types-included-with-solr.html](https://lucene.apache.org/solr/guide/6_6/field-types-included-with-solr.html)



# Solutions:

# Discussion Lab: Queries- Scalar, Non-scalar



scalars

